Description | Editorial | Solutions | Submissions

472. Concatenated Words

Solved

Hard | Topics | Companies

Given an array of strings `words` (**without duplicates**), return *all the **concatenated words** in the given list of* `words`.

A **concatenated word** is defined as a string that is comprised entirely of at least two shorter words (not necessarily distinct) in the given array.

Example 1:

```
Input: words =
["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"
]
Output:
["catsdogcats","dogcatsdog","ratcatdogcat"
]
Explanation: "catsdogcats" can be
concatenated by "cats", "dog" and "cats";
"dogcatsdog" can be concatenated by "dog",
"cats" and "dog";
"ratcatdogcat" can be concatenated by
"rat", "cat", "dog" and "cat".
```

Example 2:

```
Input: words = ["cat","dog","catdog"]
Output: ["catdog"]
```

4.1K | 77 | 9 Online

Code

C++ | Auto

```cpp
#include <vector>
#include <string>
#include <unordered_set>

using namespace std;

class Solution {
public:
    bool canForm(string word, unordered_set<string>& st) {
        int n = word.size();
        vector<bool> dp(n + 1, false);
        dp[0] = true;

        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && st.count(word.substr(j, i - j))) {
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[n];
    }

    vector<string> findAllConcatenatedWordsInADict(vector<string>& words) {
        unordered_set<string> st(words.begin(), words.end());
        vector<string> result;

        for (string word : words) {
            st.erase(word);

            if (canForm(word, st)) {
```

Saved | Ln 1, Col 1

Testcase | Test Result

Description | Editorial | Solutions | Submissions

472. Concatenated Words

Solved

Hard · Topics · Companies

Given an array of strings `words` (**without duplicates**), return *all the* ***concatenated words*** *in the given list of* `words`.

A **concatenated word** is defined as a string that is comprised entirely of at least two shorter words (not necessarily distinct) in the given array.

Example 1:

```
Input: words =
["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"]
Output:
["catsdogcats","dogcatsdog","ratcatdogcat"]
Explanation: "catsdogcats" can be concatenated by "cats", "dog" and "cats";
"dogcatsdog" can be concatenated by "dog", "cats" and "dog";
"ratcatdogcat" can be concatenated by "rat", "cat", "dog" and "cat".
```

Example 2:

```
Input: words = ["cat","dog","catdog"]
Output: ["catdog"]
```

4.1K · 77 · 8 Online

Code

C++ · Auto

```cpp
    vector<string> findAllConcatenatedWordsInADict(vector<string>& words) {
        unordered_set<string> st(words.begin(), words.end());
        vector<string> result;

        for (string word : words) {
            st.erase(word);

            if (canForm(word, st)) {
                result.push_back(word);
            }

            st.insert(word);
        }

        return result;
```

Saved · Ln 1, Col 1

Testcase | Test Result

Case 1 · Case 2

Input

words =

`["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"]`

Output

`["catsdogcats","dogcatsdog","ratcatdogcat"]`

Expected

`["catsdogcats","dogcatsdog","ratcatdogcat"]`